

SEMANA 9 – SERVIDORES Y NODE.JS

Actividad: Servidores, Node.js, Express, puertos, logs y estabilidad

<https://github.com/dannysj0214-commits/Programadores-para-la-paz/tree/fac70456fe8a87697e7f0c3e872ed95173662d8/semana9>

1. Preguntas de selección múltiple

Pregunta 1:

¿Qué es un servidor en una aplicación web?

Respuesta: B. Un sistema que ejecuta aplicaciones y responde solicitudes de los usuarios

Pregunta 2:

¿Qué significa que un servidor escuche en un puerto?

Respuesta: B. Que el servidor está esperando solicitudes en una dirección específica

Pregunta 3:

¿Cuál de los siguientes elementos es importante para la estabilidad de un servidor?

Respuesta: B. Monitorear ejecución y registrar errores

Pregunta 4:

¿Qué son los logs en un servidor?

Respuesta: A. Archivos que registran la actividad y eventos del sistema

2. Código del servidor (server.js)

javascript

```
const express = require('express');
```

```
const app = express();
```

```
app.get('/', (req, res) => {
```

```
  res.send('Servidor activo y funcionando');
```

```
});
```

```
app.get('/estado', (req, res) => {
```

```
  res.json({
```

```
    estado: "Servidor funcionando",
```

```
servicio: "API comunitaria"

});

});

app.listen(3000, () => {

  console.log('Servidor ejecutándose en puerto 3000');

});
```

Ejecución: node server.js

Pruebas en navegador:

- <http://localhost:3000> → "Servidor activo y funcionando"
- <http://localhost:3000/estado> → {"estado":"Servidor funcionando","servicio":"API comunitaria"}

3. Ejecución del servidor

¿Qué ocurre cuando ejecutas node server.js?

Al ejecutar "node server.js", el motor de JavaScript (Node.js) lee el archivo, carga las dependencias (Express), crea una instancia del servidor y lo pone en funcionamiento. El servidor queda activo y "escuchando" en el puerto 3000, esperando solicitudes entrantes.

¿Qué mensaje aparece en la terminal?

Cuando el servidor inicia correctamente, aparece el mensaje:
"Servidor ejecutándose en puerto 3000"

¿Qué sucede cuando visitas la ruta /estado?

Al visitar <http://localhost:3000/estado>, el servidor responde con un objeto JSON:

```
json

{

  "estado": "Servidor funcionando",

  "servicio": "API comunitaria"

}
```

4. Componentes de un entorno servidor

1. Sistema Operativo

El sistema operativo (ej. Linux, Windows, macOS) es la capa base sobre la que se ejecuta todo el

software del servidor. Gestiona los recursos del hardware y proporciona los servicios fundamentales para que Node.js y las aplicaciones puedan ejecutarse.

2. Node.js

Node.js es un entorno de ejecución de JavaScript del lado del servidor. Permite ejecutar código JavaScript fuera del navegador, utilizando el motor V8 de Google. Es la tecnología que interpreta y ejecuta el archivo `server.js`.

3. Aplicación `server.js`

`server.js` es el archivo que contiene el código fuente de la aplicación servidora. Define las rutas, la lógica de respuesta y la configuración del servidor (puerto). Es el "cerebro" de la aplicación.

4. Puerto (3000)

El puerto es un número que identifica un canal de comunicación específico dentro de un servidor. Es donde el servidor "escucha" las solicitudes entrantes. Es como el "número de oficina" dentro de un edificio (el servidor).

5. Logs

Los logs son registros cronológicos de eventos que ocurren en el servidor. Almacenan información como fecha y hora de cada petición, dirección IP del cliente, ruta solicitada, errores y tiempos de respuesta. Son fundamentales para monitoreo, auditoría y resolución de problemas.

5. Reflexión de la semana

¿Por qué es importante que las plataformas tecnológicas que usan las comunidades funcionen de manera estable y segura en un servidor?

La estabilidad y seguridad de un servidor son aspectos fundamentales para cualquier plataforma comunitaria porque de ellas depende la confianza de los usuarios y la continuidad de los procesos participativos. Cuando una comunidad utiliza una plataforma digital para organizarse, comunicarse o tomar decisiones colectivas, deposita en ese sistema su tiempo, sus datos y la legitimidad del proceso. Si el servidor falla constantemente, los usuarios pierden la confianza y abandonan la plataforma, lo que puede llevar al fracaso de la iniciativa comunitaria.

Un servidor inestable puede causar caídas inesperadas, tiempos de respuesta lentos y pérdida de información, lo que afecta directamente la experiencia del usuario. En momentos críticos, como una votación o una asamblea virtual, una caída del servidor puede invalidar todo el proceso y generar frustración en los participantes. Por eso, el monitoreo constante y la generación de logs son prácticas esenciales que permiten detectar problemas a tiempo y prevenir fallos mayores.

La seguridad, por su parte, protege los datos personales de los usuarios y evita accesos no autorizados. Una plataforma comunitaria maneja información sensible que debe ser resguardada con prácticas como el cifrado de conexiones, el control de accesos y la actualización periódica del software. Una violación de seguridad puede exponer datos privados, dañar la reputación de la comunidad y generar desconfianza entre los participantes.

El uso de herramientas como Node.js y Express permite construir servidores ligeros y eficientes, pero su correcto funcionamiento depende de buenas prácticas de configuración y mantenimiento. Comprender cómo funciona el servidor, qué hace cada componente y cómo se monitorea su actividad es el primer paso para garantizar una plataforma confiable.

En conclusión, una plataforma comunitaria que funciona de manera estable y segura no solo cumple con un estándar técnico, sino que demuestra respeto hacia las personas que confían en ella. La estabilidad y la seguridad son condiciones necesarias para que la tecnología esté al servicio de la participación ciudadana y no se convierta en un obstáculo para ella.

Fecha de entrega: 26/06/2026